

Reviewer Pack — Minimal Number of Simplicial Generators for Boolean Satisfiability

Entry fad8caedb5f4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 22:26:23 UTC

Minimal Number of Simplicial Generators for Boolean Satisfiability

Entry ID: fad8caedb5f4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 22:26:23 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Simplicial Complexes) **Field B** (complexity object): Boolean Satisfiability (CNF Formulas)

Statement:

For any CNF formula with n variables, the minimal number of simplicial generators required to construct a topological realization that can be used to verify the satisfiability of the formula is bounded by $O(n^{\{1.5\}})$.

Rationale (proposer's reasoning):

Simplicial complexes are used in algebraic topology to study the structure of spaces. By connecting the properties of simplicial complexes with the verification process of CNF formulas, this conjecture aims to expose a connection that could lead to new algorithms for SAT solving.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1f20bf1b1bbc35dc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture holds if the number of simplicial generators for a CNF formula with n variables is between $O(n)$ and $O(n^{\{1.5\}})$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "simplicial complex" AND "Boolean satisfiability" AND CNF - "algebraic topology" AND simplicial generator AND "CNF formula" - "topological realization" AND $O(n^{\{1.5\}})$ AND Boolean satisfiability

Top relevant hits considered: - [http://arxiv.org/abs/1411.5306v3] The Simplicial EHP Sequence in A1-Algebraic Topology - [http://arxiv.org/abs/2309.11304v3] A new quantum computational set-up for algebraic topology via simplicial sets - [http://arxiv.org/abs/2306.16951v1] Applying language models to algebraic topology: generating simplicial cycles using multi-labeling in Wu's formula - [http://arxiv.org/abs/1110.2370v3] The Arone-Goodwillie spectral sequence for $\Sigma^\infty \Omega^n$ and topological realization at odd primes - [http://arxiv.org/abs/2404.01236v2] Cosmic topology. Part IVa. Classification of manifolds using machine learning: a case study with small toroidal universe

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
            if random.choice([True, False]):
                clause[1] *= -1
            cnf.append(clause)
        return cnf

    def is_satisfiable(cnf):
        # Simple backtracking solver for CNF
        assignment = [None] * (n + 1)

        def backtrack(i):
            if i > n:
                return True
            for val in [-1, 1]:
                assignment[i] = val
                if all(any(assignment[abs(lit)] == l for l in clause) for clause in cnf):
                    if backtrack(i + 1):
                        return True

        return backtrack(1)
```

```

        assignment[i] = None
        return False

    return backtrack(1)

def construct_simplicial_complex(cnf):
    # Construct a simplicial complex from the CNF formula
    vertices = set()
    for clause in cnf:
        vertices.update(abs(lit) for lit in clause)

    simplices = []
    for i in range(len(vertices)):
        simplices.append([list(vertices)[i]])

    return simplices, len(vertices)

n = random.randint(5, 40)
m = random.randint(n * 2, n * 3)
cnf = generate_cnf(n, m)

if not is_satisfiable(cnf):
    return {
        "metric_name": "num_simplicial_generators",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "CNF formula is not satisfiable"
    }

simplices, num_vertices = construct_simplicial_complex(cnf)

return {
    "metric_name": "num_simplicial_generators",
    "metric_value": len(simplices),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": len(simplices) <= n ** 1.5,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a676d5a5.py", line 92, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a676d5a5.py", line 65, in run_trial
    if not is_satisfiable(cnf):
           ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a676d5a5.py", line 47, in is_satisfiable
    return backtrack(1)
           ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a676d5a5.py", line 41, in backtrack
    if all(any(assignment[abs(lit)] == l for l in clause) for clause in cnf):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a676d5a5.py", line 41, in <genexpr>
    if all(any(assignment[abs(lit)] == l for l in clause) for clause in cnf):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a676d5a5.py", line 41, in <genexpr>
    if all(any(assignment[abs(lit)] == l for l in clause) for clause in cnf):
           ^^^
NameError: name 'lit' is not defined. Did you mean: 'List'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing any data that could be used to evaluate the conjecture. | next: Review and fix the error in the test code to ensure it can run without crashing and produce results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20089
2	preregistration	ollama_remote	glm4:latest	0	0	9994
3	novelty	ollama_remote	glm4:latest	0	0	11397
4	novelty	ollama_remote	glm4:latest	0	0	15261
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17611
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19257
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11378
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16005
9	judge	ollama_remote	glm4:latest	0	0	9007

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129998 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/fad8caedb5f4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fad8caedb5f4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fad8caedb5f4.tar.gz` (if generated)